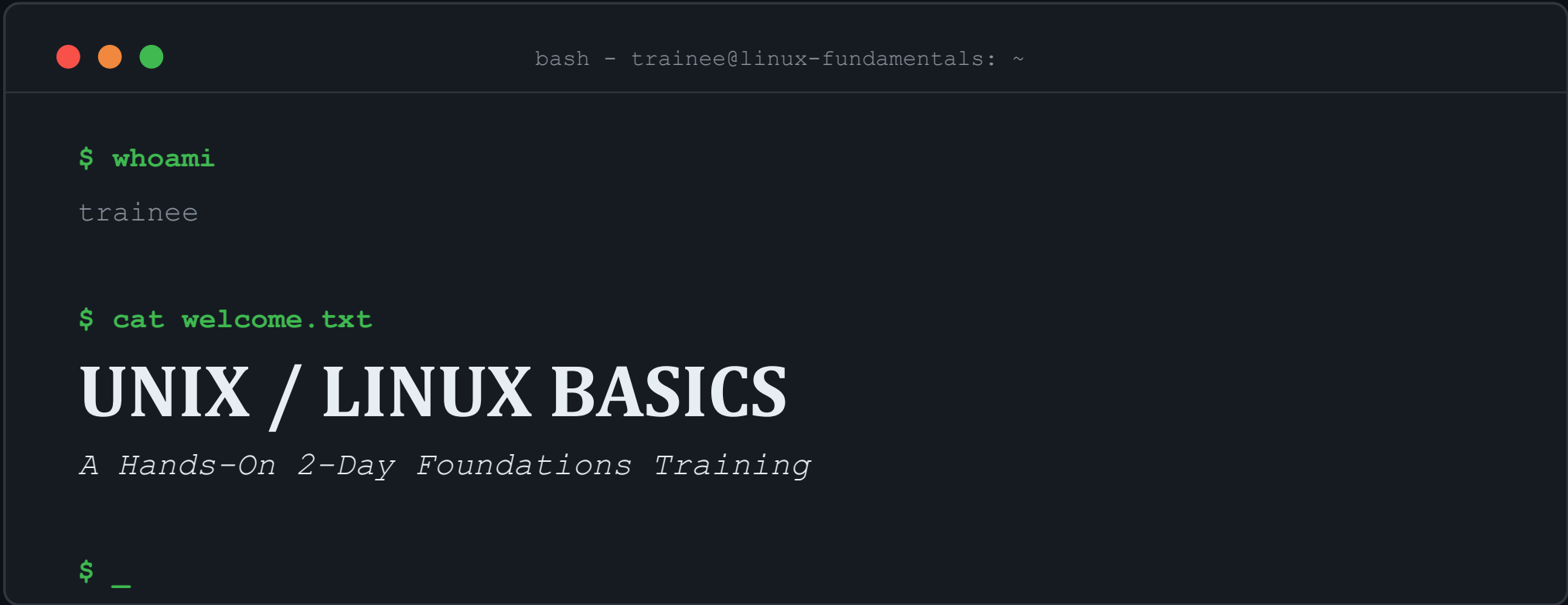




```
bash - trainee@linux-fundamentals: ~  
  
$ whoami  
trainee  
  
$ cat welcome.txt  
  
UNIX / LINUX BASICS  
A Hands-On 2-Day Foundations Training  
  
$ _
```

Command Line

File System

Permissions

vi Editor

Filters & Regex

Shell Scripting



Who This Course Is For

COURSE DESCRIPTION

This course is for anyone who will be using or administering a Linux/UNIX system. Casual users will find it sufficient on its own; programmers and system administrators will find it an effective foundation for further, more advanced study.



PREREQUISITE

Exposure to any multi-user operating environment (e.g. Windows, macOS, or another Unix-like system) is helpful but not required.

What You'll Be Able To Do



Understand the history of UNIX and the Linux family of distributions



Explain what the Linux kernel does and how multi-user, multi-tasking works



Log in, manage a session, and get help using man pages



Edit files confidently using the vi text editor



Navigate the file system and use core UNIX tools & commands



Execute commands and pipelines from the command line



Use regular expressions to search and match text patterns



Apply UNIX power tools (filters) to slice and reshape text



Write basic shell scripts (bash, Korn, or C shell)



Use sed and awk for stream editing and text reporting



Manage processes and schedule recurring tasks



Perform basic Linux system administration, incl. permissions

DAY 1

Foundations - Getting Comfortable with Linux



MODULE 01

Introduction to Linux



MODULE 02

The Linux File System



MODULE 03

File System Security



MODULE 04

The vi Text Editor

Introduction to Linux

Where Linux comes from, how it's built, and how to start a session.

WHAT YOU'LL LEARN IN THIS MODULE

History

Kernel

Architecture

Login/Logout

man pages





A Short History & the Linux Family

- ▶ UNIX was born in the late 1960s at Bell Labs; Linux was created in 1991 by Linus Torvalds as a free, UNIX-like kernel.
- ▶ There are many "different kinds" of Linux - distributions such as Ubuntu, Red Hat / CentOS, Debian, and openSUSE.
- ▶ All distributions share the same Linux kernel; they differ in packaging, defaults, and tools bundled on top.
- ▶ Linux OS architecture, from the bottom up: hardware → kernel → shell → applications & utilities.





The Kernel, Multi-tasking & Multi-user

The Kernel

- ▶ The core of the operating system
- ▶ Manages the CPU, memory, storage, and devices
- ▶ The shell and every tool talk to hardware through it

Multi-tasking & Multi-user

- ▶ Many users can be logged in and working at the same time
- ▶ Many processes run concurrently
- ▶ The kernel schedules who/what gets the CPU next



Logging In & Managing Your Session

Session Basics

- ▶ Log in with a username and password
- ▶ Log out cleanly when you're done
- ▶ Linux keyboard anomalies - Backspace/Delete and Ctrl combos can vary by terminal

Passwords

- ▶ Change your password regularly with `passwd`
- ▶ Portable password rules: sufficient length, mixed case, numbers & symbols
- ▶ Avoid dictionary words and reused passwords



Terminals & Finding Help

- ▶ "Dumb" text terminals vs. graphical X-Windows terminals - both give you a shell prompt.
- ▶ `man <command>` opens the built-in manual page for any command.
- ▶ Manual pages are organized into numbered sections (commands, system calls, files, etc.).
- ▶ `whatis` and `info` offer additional ways to search and browse documentation.

Good habit: "when in doubt, man it out" — the manual is always closer than a web search.



Q1. Which component of Linux directly manages hardware resources such as the CPU, memory, and devices?

A Shell

B Kernel

C A man page

D Terminal emulator



Q1. Which component of Linux directly manages hardware resources such as the CPU, memory, and devices?

A Shell

B Kernel



C A man page

D Terminal emulator

Answer: B - The kernel talks directly to hardware and manages resources; the shell is just the user's command interface on top of it.

The Linux File System

How files and directories are organized, named, and navigated.

WHAT YOU'LL LEARN IN THIS MODULE

Paths

Directory Tree

Core Commands

find & locate

Links





File System Fundamentals

● Role & Naming

- ▶ The file system organizes everything as files and directories
- ▶ Rules for naming files - case-sensitive, avoid special/reserved characters

● Finding Your Way

- ▶ Absolute pathname: the full path starting from /
- ▶ Relative pathname: relative to where you currently are
- ▶ Working directory (pwd) vs. home directory (~)



File System Geography

/

Root — top of the entire tree

/bin

Essential command binaries

/etc

System configuration files

/home

Personal user directories

/var

Variable data — logs, mail, spool

/usr

Installed programs & libraries

/tmp

Temporary files, cleared on
reboot

/dev

Device files for hardware



Core Navigation & File Commands

~/ - core commands			
<code>pwd</code>	Print working directory	<code>mv</code>	Move or rename files
<code>cd</code>	Change directory	<code>rm</code>	Remove files
<code>ls</code>	List directory contents	<code>mkdir</code>	Create a directory
<code>cat</code>	Display / create files	<code>rmdir</code>	Remove an empty directory
<code>cp</code>	Copy files		



Viewing, Comparing & Finding Files

● Viewing & Comparing

- ▶ more / less - page through long files
- ▶ file - identify a file's type
- ▶ cmp / comm / diff - compare two files

● Shortcuts & Search

- ▶ . current directory, .. parent directory, ~ home directory
- ▶ find - search for files by name, type, size, and more
- ▶ locate + updatedb - fast, indexed filename search
- ▶ ln - create hard and symbolic links



Q2. Which path always starts from the root directory, regardless of where you currently are?

A A relative pathname

B An absolute pathname

C The ~ shortcut alone

D The working directory



Q2. Which path always starts from the root directory, regardless of where you currently are?

A A relative pathname

B An absolute pathname



C The ~ shortcut alone

D The working directory

Answer: B - Absolute paths start with / and resolve the same way from anywhere; relative paths depend on your current working directory.

File System Security

Users, groups, and the permission model that protects every file.

WHAT YOU'LL LEARN IN THIS MODULE

Users & Groups

rwX

chmod

umask

Sticky Bit



3



Two-Level Security & Users

The Security Model

- ▶ Two-level structure: who you are, and what you're allowed to do
- ▶ Every file recognizes three categories: owner, group, and others

Groups

- ▶ Users are organized into groups
- ▶ Group membership grants shared access to files
- ▶ A user can belong to more than one group



Reading a Permission String



- ▶ r / w / x on a file: read its contents / modify it / run it as a program
- ▶ r / w / x on a directory: list contents / create-delete entries / "enter" (cd into) it
- ▶ The first character marks the type: "-" for a regular file, "d" for a directory, "l" for a link



Changing Permissions & Ownership

~/ — ownership commands

chmod	Change permissions - symbolically (u+x) or numerically (755)
chown	Change a file's owner
chgrp	Change a file's group owner
root	The super-user - bypasses normal permission checks



Defaults, Timestamps & the Sticky Bit

- ▶ `umask` sets the default permissions applied to every new file and directory you create.
- ▶ Every file tracks three timestamps: last access, last modification, and last (metadata) change.
- ▶ "Sticky" directories, like `/tmp`, let anyone create files - but only the owner can delete or rename their own file inside.





Q3. A file has permissions -rwxr--r--. What can members of the file's group do?

A Read, write, and execute

B Read only

C Write only

D Nothing at all



Q3. A file has permissions `-rwxr--r--`. What can members of the file's group do?

A Read, write, and execute

B Read only



C Write only

D Nothing at all

Answer: B — Reading the string as owner=rwx, group=r--, others=r--, group members only have read access.

The vi Text Editor

The universal, keyboard-driven editor found on every Unix system.

WHAT YOU'LL LEARN IN THIS MODULE

Modes

Yank & Put

Search

Substitution

.exrc



4



Why vi? Understanding the Modes

Why Learn vi

- ▶ Available on virtually every Unix/Linux system, even minimal ones
- ▶ Fast and fully keyboard-driven - no mouse required

The Modes

- ▶ Command mode - move around and issue commands
- ▶ Insert mode - type text into the file
- ▶ Ex (last-line) mode - save, quit, search & replace
- ▶ `:set showmode` displays which mode you're in



Editing, Yanking & Putting

~/notes.txt - vi (command mode)

.

Repeat the last change

~

Toggle the case of a character

u

Undo the last change

yy / dd

Yank (copy) or delete a whole line

J

Join the current line with the next

p / P

Put (paste) after / before the cursor



Searching & Substitution

Search

- ▶ `/pattern` and `?pattern` search forward / backward
- ▶ `n / N` repeat the last search - same or opposite direction
- ▶ Metacharacters `^ $ . * \ [ ]` build the pattern

Substitute

- ▶ `:s/old/new/` - replace on the current line
- ▶ `:%s/old/new/g` - replace every match in the file
- ▶ `:10,20s/old/new/` - replace within a line range

Customizing vi



- ▶ `:set` toggles editor options - line numbers, wrap, ignorecase, and more.
- ▶ `.exrc` is a startup file that pre-loads your favorite vi settings automatically.
- ▶ Split windows let you view or edit multiple files side by side.
- ▶ Visual mode lets you mark and select text for block operations.





Q4. In vi, which mode must you be in to type and insert new text into the document?

A Command mode

B Insert mode

C Ex (last-line) mode

D Visual mode



Q4. In vi, which mode must you be in to type and insert new text into the document?

A Command mode

B Insert mode



C Ex (last-line) mode

D Visual mode

Answer: B — Command mode is for navigation and commands; pressing i, a, or o switches into Insert mode so you can type text.

DAY 2

Power Tools - Text, Processes & Automation



MODULE 05

Filters & Text Processing



MODULE 06

Regular Expressions



MODULE 07

Processes & Task Scheduling



MODULE 08

Shell Scripting & Power Tools

Filters & Text Processing

Small, composable commands that read, transform, and reshape text.

WHAT YOU'LL LEARN IN THIS MODULE

`split/csplit`

`wc/head/tail`

`cut/paste/join`

`sort/tr`





What Is a Filter?

- ▶ A filter reads input, transforms it, and writes output - built to be chained together with a pipe: `cmd1 | cmd2`.
- ▶ `split` and `csplit` break one large file into multiple smaller files.
- ▶ Filters are the building blocks behind powerful one-line Unix workflows.



wc, head & tail

~/ - previewing files

wc	Count lines, words, and characters in a file
head	Display the beginning of a file
tail	Display the end of a file (tail -f follows live updates)



cut, paste & join

~/ - reshaping columns

cut	Split a file vertically - extract specific columns or fields
paste	Merge lines from multiple files side by side
join	Combine two files based on a shared field



sort & tr

~/ - ordering & translating

sort

Sort file contents - alphabetic, numeric, reverse, or by field

tr

Translate or delete characters from the input stream



Q5. Which command would you use to display only the last 10 lines of a log file?

A head

B tail



C cut

D wc

Answer: B — tail shows the end of a file (last 10 lines by default); head shows the beginning instead.

Regular Expressions

Pattern matching that powers search-and-replace across the system.

WHAT YOU'LL LEARN IN THIS MODULE

Metacharacters

grep

egrep

fgrep





Regex Metacharacters at a Glance

^

Matches the start of a line

\$

Matches the end of a line

.

Matches any single character

*

Zero or more of the previous character

[]

Matches any one character in the set

\

Escapes a metacharacter to match it
literally



grep, egrep & fgrep

~/ — the grep family

grep

Search for a pattern using basic regular expressions

egrep

Extended regex search — supports +, ?, | without escaping

fgrep

Fixed-string search — fast matching of multiple literal strings



Q6. Which command searches using extended regular expressions (supporting +, ?, and | without escaping)?

A fgrep

B grep

C egrep



D sed

Answer: C — egrep (grep -E) understands extended regex metacharacters directly; fgrep matches fixed, literal strings only.

Processes & Task Scheduling

What's running on the system, and how to schedule work for later.

WHAT YOU'LL LEARN IN THIS MODULE

`ps/top/kill`

`Foreground/Background`

`at`

`cron/crontab`





Understanding Processes

Processes

- ▶ The shell itself runs as a process
- ▶ Every process has a parent process — except one

init

- ▶ init is the very first process started at boot (PID 1)
- ▶ Every other process descends from it



Managing Running Processes

~/ - process control

`& / fg / bg`

Run in background; bring to foreground;
resume in background

`kill`

Terminate a process by its PID

`ps / pstree`

List processes / view them as a tree

`nice`

Start a process with adjusted priority

`jobs / top`

List shell jobs / live resource monitor

`sleep / wait /
nohup`

Pause; wait for a process; survive logout



Scheduling with at

● The at Command

- ▶ Runs a command once, at a specified future time
- ▶ Time definitions: now, midnight, teatime, or an explicit time/date

● Managing at Jobs

- ▶ `at -l / atq` — list pending jobs
- ▶ `at -r / atrm` — remove a pending job
- ▶ Security: only permitted users can schedule jobs (`at.allow / at.deny`)



cron & crontab

~/ - recurring jobs

<code>cron</code>	The background daemon that runs recurring jobs on a schedule
<code>crontab -e/-l/-r</code>	Edit, list, or remove your personal cron table
<code>min hour day mon wk</code>	The 5-field schedule format that precedes every cron command
<code>batch</code>	Runs a queued job once system load allows



Q7. Which command schedules a job to run repeatedly at fixed intervals, such as every day at 2 AM?

A at

B nohup

C crontab



D nice

Answer: C — cron (managed with crontab) handles recurring scheduled jobs; at schedules a single one-time job instead.

Shell Scripting & Power Tools

Automating work with scripts, sed, and awk.

WHAT YOU'LL LEARN IN THIS MODULE

`bash/ksh/csh`

`sed`

`awk`





Shell Scripting Basics

Choosing a Shell

- ▶ bash — the most common default shell on Linux
- ▶ ksh (Korn shell) and csh (C shell) — common alternatives

Why Script?

- ▶ Automate repetitive sequences of commands
- ▶ Starts with `#!/bin/bash` and holds variables, conditionals & loops



sed — the Stream Editor

What sed Does

- ▶ Edits text as it streams through, line by line
- ▶ No interactive file opening required
- ▶ Perfect for scripted, repeatable text transformations

```
sed examples

$ sed 's/foo/bar/' file.txt

# replaces the first "foo" per line

$ sed -i 's/foo/bar/g' file.txt

# edits the file in place, all matches
```



awk — Pattern Scanning & Reporting

What awk Does

- ▶ Splits each line into fields: \$1, \$2, \$3 ... and \$0 for the whole line
- ▶ Pairs a pattern with an action: pattern { action }
- ▶ Ideal for column reports, sums, and quick data extraction

```
awk examples

$ awk '{print $1, $3}' file.txt

# print the 1st and 3rd column

$ awk -F, '{sum+=$2} END{print sum}' f.csv

# sum the 2nd comma-separated column
```



Q8. Which tool is best suited for extracting and reporting on specific columns of text, using \$1, \$2 field references?

A sed

B awk



C grep

D vi

Answer: B — awk is a pattern-scanning language built around fields (\$1, \$2...) and records — ideal for column-based reporting.

Key Takeaways



DAY 1

- ▶ Linux architecture, the kernel, and starting/ending a session
- ▶ Navigating the file system with absolute & relative paths
- ▶ The permission model: users, groups, rwx, and chmod
- ▶ Editing files confidently with vi

DAY 2

- ▶ Chaining filter commands to slice and reshape text
- ▶ Matching patterns with regular expressions & the grep family
- ▶ Inspecting and controlling processes; scheduling with cron
- ▶ Automating work with shell scripts, sed, and awk



```
$ logout
```

THANK YOU

Questions? Keep practicing on the command line — it sticks best by doing.